

2026 年南京林业大学校内电子设计竞赛

基于 TI MSPM0 的智能小车控制装置设计报告

作品名称：智能循迹与图像识别小车

参赛队员：已脱敏

指导教师：已脱敏

所在学院：信息科学技术学院、人工智能学院

完成日期：2026 年 5 月

摘要

本作品面向 2026 年南京林业大学校内电子设计竞赛 D 题“基于 TI MSPM0 的智能小车控制装置”，完成了基于 MSPM0G3507 主控的循迹控制系统和基于 K230 摄像头平台的图像识别模块。循迹部分使用 8 路灰度/红外传感器采集赛道黑线状态，结合编码器测速、IMU 航向角反馈、按键模式选择和 OLED 状态显示，实现了题目路线的循迹绕行，并完成 1、2、3、4、5 圈设定运行及到圈停车。图像识别部分完成了红、绿、蓝颜色识别，圆形、矩形、三角形识别以及七段数码管数字识别，识别结果能够在 LCD/IDE 画面中实时标注，整体效果较好。

关键词：智能小车；循迹控制；MSPM0G3507；K230；图像识别；PID

一、设计方案工作原理

1.1 赛题要求与完成情况

赛题要求必须采用 TI MSPM0 系列 MCU 作为巡迹和电机控制主控，完成标准赛道自动循迹、障碍物自主避让、定点精准启停，并在行驶过程中完成颜色、形状和数字识别。标准赛道为白色哑光底面、黑色哑光循迹线，线宽 18~20 mm，要求按照 C-E-G-A-B-C-D-A-B 路线完成一圈。根据当前源码、图片和调试结果，本作品已完成 MSPM0 主控循迹框架、1~5 圈设定循迹及到圈停车、K230 视觉识别；避障、二维云台对准、障碍物距离显示和终点 2 cm 精准泊车尚未形成可验证结果。

赛题任务	要求摘要	当前完成情况
主控要求	必须采用 TI MSPM0 系列 MCU 负责巡迹与电机控制	已采用 MSPM0G3507
自动循迹一圈	按 C-E-G-A-B-C-D-A-B 路线走完一圈， $t \leq 5\text{ s}$	可完成一圈，时间待实测
圈数设定	N 可在 1~5 之间设定， $t \leq 20\text{ s}$	1、2、3、4、5 圈均可设定运行并停车，时间待实测
障碍物避让	检测障碍后减速、停车、绕行并回归轨迹	未形成完整结果
状态显示	显示速度、障碍物距离、循迹状态	OLED 已显示 mode/set/yaw，速度和距离待完善
颜色识别	识别红绿蓝并实时显示，蜂鸣器提示，云台对准	颜色识别完成，蜂鸣器/云台联动待完善
形状与数字识别	识别圆形、方形、三角形及 0~9 数字	圆/矩形/三角形/七段数字识别完成较好
终点停车	支持一键启停、紧急停车，停车偏差 $\leq 2\text{ cm}$	支持按键启停，精准泊车待实测

表 1-1 赛题要求与作品完成情况

1.2 技术方案分析比较

循迹部分采用“传感器离散状态判断 + IMU 航向角闭环 + 双电机 PWM 差速控制”的方案。与单纯依靠灰度传感器开环转向相比，该方案能够在弯道、交叉点和短暂丢线时利用航向角约束

车身方向；与完全视觉循迹相比，该方案对算力和光照条件要求较低，调试链路更短，更适合在竞赛现场快速修正参数。

图像识别部分采用 K230 平台直接调用图像库接口完成阈值分割、连通域分析、圆形/矩形检测和七段区域统计。该方案不依赖离线训练模型，代码参数直观，便于根据现场光照调整 LAB 阈值和面积阈值。

1.3 系统结构与工作原理

层级	组成	作用
感知层	8 路循迹传感器、编码器、IMU、K230 摄像头	采集赛道、速度、姿态和图像信息
控制层	MSPM0G3507 主控、K230 图像处理程序	完成循迹决策、PID 运算和视觉识别
执行层	左右直流电机、蜂鸣器、OLED/LCD 显示	输出行驶动作、状态提醒和识别结果
软件层	Keil C 工程、K230 Python 脚本	组织初始化、中断控制、识别算法和人机交互

表 1-2 系统总体结构

系统启动后，MSPM0G3507 先完成时钟、OLED、IMU、按键、电机、编码器、串口、循迹传感器和定时器初始化。主循环负责按键模式切换、姿态数据解析和 OLED 状态显示；10 ms 定时器中断读取编码器并根据当前模式调用 track1、track2、track3 或 track4。K230 视觉端独立运行摄像头采集和图像处理程序，在显示画面上叠加识别框、类别与数字结果。赛题中要求的障碍物避让和二维云台联动可以在该架构上继续扩展，但当前初稿只按已完成与可验证的功能撰写。

二、核心部件电路设计

2.1 主控与传感器接口

小车控制核心为 MSPM0G3507，工程文件中以 Keil/uVision 方式组织，主程序位于 user/main.c。循迹传感器 D1 至 D8 分别接入 PB13、PB15、PA31、PA28、PB1、PB4、PB17、PB12，并在 xunji_init() 中配置为上拉输入。传感器数字量经过 digital() 统一读取，黑线检测结果用于判断车体相对赛道中心线的偏移。

```
void xunji_init()
{
    gpio_init(GPIOB, DL_GPIO_PIN_13, PB13, IN_UP);    // D1
    gpio_init(GPIOB, DL_GPIO_PIN_15, PB15, IN_UP);    // D2
    gpio_init(GPIOA, DL_GPIO_PIN_31, PA31, IN_UP);     // D3
    gpio_init(GPIOA, DL_GPIO_PIN_28, PA28, IN_UP);     // D4
    gpio_init(GPIOB, DL_GPIO_PIN_1,  PB1,  IN_UP);     // D5
    gpio_init(GPIOB, DL_GPIO_PIN_4,  PB4,  IN_UP);     // D6
    gpio_init(GPIOB, DL_GPIO_PIN_17, PB17, IN_UP);     // D7
    gpio_init(GPIOB, DL_GPIO_PIN_12, PB12, IN_UP);     // D8
}
```

图 2-1 8 路循迹传感器初始化程序片段

2.2 电机、编码器与姿态反馈

左右电机通过 PWM 占空比控制速度和方向，编码器用于获得左右轮当前速度，IMU 提供 yaw 航向角。控制程序采用角度环与速度环结合的思路：角度环根据目标航向与当前航向计算转向修正量，速度环根据左右编码器反馈修正 PWM 输出。这样能够在巡线时保持基本前进速度，同时通过差速补偿方向误差。

```
void turn_pid(int base, int target)
{
    angle.now = yaw_angle_int;
    angle.target = target;
    pid_cal(&angle);
    motorA.now = left_encoder;
    motorA.target = base - angle.out;
    motorB.now = right_encoder;
    motorB.target = base + angle.out;
    pid_cal(&motorA);
    pid_cal(&motorB);
    pid_out_limit(&motorA);
    pid_out_limit(&motorB);
    Set_left_pwm((int)motorA.out);
    Set_right_pwm((int)motorB.out);
}
```

图 2-2 航向角闭环与左右轮差速控制程序片段

2.3 K230 视觉识别平台

视觉识别部分采用 K230 摄像头与 LCD/IDE 显示输出。程序设置 400×240 图像采集分辨率，显示端采用 640×480 LCD 模式，并通过 MediaManager 管理摄像头和显示资源。颜色识别使用 LAB 阈值区分红、绿、蓝，形状识别调用 find_circles、find_rects 及自定义三角形检测流程，七段数码管识别则把数字外框划分为 A 至 G 七个区域，根据亮段组合匹配 0 至 9。

三、系统软件设计分析

3.1 系统总体工作流程

小车控制软件按“初始化—等待按键—模式运行—实时闭环—状态显示”的流程工作。mode 变量用于选择任务模式，set 变量用于启动运行；按键 1 循环切换模式，按键 2 确认启动。定时器中断以固定周期调用对应循迹函数，使传感器采集、编码器读取和 PID 计算在稳定节拍下执行。

```
if (KeyNum == 1) {
    mode++;
    if (mode == 5) { mode = 1; }
    if (set == 1) { set = 0; }
}
if (KeyNum == 2) {
    set = 1;
}
```

```

void TIM8_IRQHandler()
{
    left_encoder = -read_encoder1();
    right_encoder = -read_encoder2();
    if (mode == 1 && set == 1) track1();
    if (mode == 2 && set == 1) track2();
    if (mode == 3 && set == 1) track3();
    if (mode == 4 && set == 1) track4();
}

```

图 3-1 模式选择与定时器任务调度程序片段

3.2 循迹控制模块

循迹函数根据 D1 至 D8 的组合状态判断偏移方向。中间 D4、D5 检测到黑线时，左右电机以相近 PWM 前进；黑线偏向左侧时，右轮增速、左轮降速，使车体向左修正；黑线偏向右侧时，左轮增速、右轮降速，使车体向右修正。对于交叉点和丢线状态，程序通过 now_statue、last_statue 和 change_flag1 记录状态变化，并在指定计数点切换目标航向角或停车。

传感器状态示例	处理策略	电机输出特征
11100111	位于中心附近	左右轮 PWM 接近，直行
11001111/10011111	黑线偏左	右轮 PWM 较大，左轮较小
11111011/11111101	黑线偏右	左轮 PWM 较大，右轮较小
11111111	短时未检测到黑线	更新状态计数，等待下一次检测
状态变化达到目标计数	切换目标航向或停车	调用 turn_pid/check 或置零 PWM

表 3-1 循迹状态与控制策略

单圈任务使用 track2/track3 类控制逻辑，在交叉点计数达到设定值后停车；圈数任务使用 track4 扩展多圈计数逻辑，根据设定圈数和交叉点状态变化切换目标航向分段。实际调试结果表明，小车已经能够完成 1、2、3、4、5 圈设定运行，并在达到设定圈数后停车，说明交叉点计数、航向角分段和停车控制链路已经闭合。后续可继续围绕 20 s 时间指标、弯道速度分配和停车位置精度进行优化。

3.3 图像识别模块

颜色识别模块遍历红、绿、蓝三个 LAB 阈值，调用 find_blobs 找到面积最大的色块，并在画面中绘制外接矩形。形状识别模块检测圆形和矩形，三角形识别模块综合轮廓和 blob 方法，去除中心点距离过近的重复结果，并按面积排序输出。七段数码管识别模块先自动框选数字区域，再统计 A、B、C、D、E、F、G 七个段位的亮像素比例，最后与标准七段编码表匹配。

```

THRESHOLDS = [
    (0, 66, 7, 127, 3, 127),      # red
    (42, 100, -128, -17, 6, 66),   # green
    (43, 99, -43, -4, -56, -7),    # blue
]

def detect_color(img):

```

```

best_color = -1
best_blob = None
for i in range(3):
    blobs = img.find_blobs([THRESHOLDS[i]], area_threshold=200,
                           merge=True, margin=10)
    if blobs:
        max_b = max(blobs, key=lambda b: b.area())
        if best_blob is None or max_b.area() > best_blob.area():
            best_blob = max_b
            best_color = i
return COLOR_NAMES[best_color] if best_blob is not None else "NONE"

```

图 3-2 K230 颜色识别核心程序片段



图 3-3 红色目标识别结果



图 3-4 绿色目标识别结果



图 3-5 蓝色目标识别结果



图 3-6 圆形目标识别结果

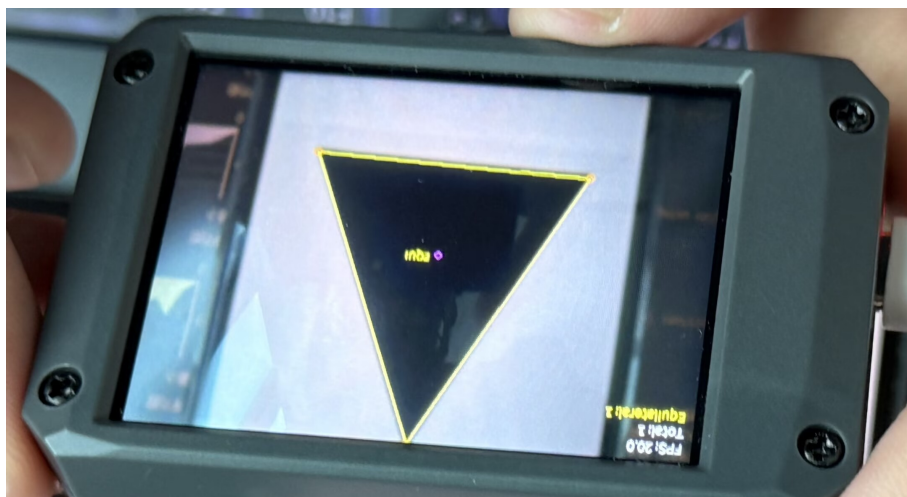


图 3-7 三角形目标识别结果

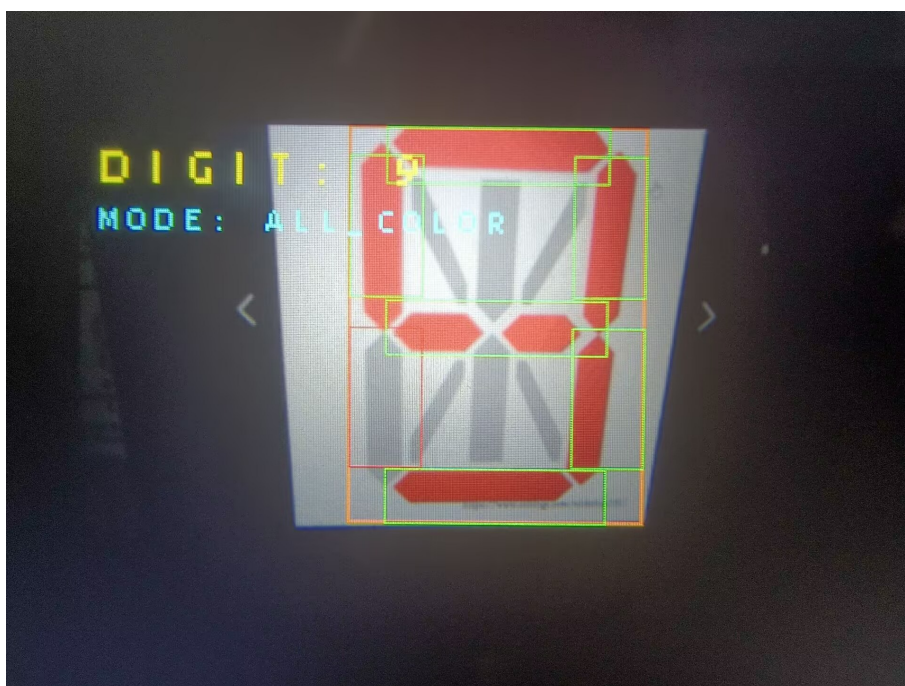


图 3-8 七段数码管识别结果

四、竞赛工作环境条件

4.1 软件环境

小车底层程序采用 C 语言开发，工程以 Keil/uVision 项目文件组织，源码包含 user、code、ml_libs 和 m0_sdk 等目录。K230 图像识别程序采用 Python 风格脚本，使用 media.sensor、media.display、media.media 和 image 模块完成摄像头、显示和图像处理。

类别	环境/文件	用途
单片机工程	user/project.uvprojx	MSPM0G3507 工程配置与编译
主控程序	user/main.c	系统初始化、按键模式、定时器调度
循迹模块	user/xunji.c / xunji.h	传感器读取、循迹状态机和停车逻辑
PID 模块	user/_pid.c / code/pid.c	电机速度环与角度环控制
视觉脚本	K230 图像识别/*.py	颜色、形状和七段数字识别

表 4-1 软件开发环境与文件分工

4.2 硬件平台与调试条件

硬件平台由小车底盘、左右电机、编码器、MSPM0G3507 主控板、8 路循迹传感器、IMU、OLED、蜂鸣器以及 K230 摄像头/显示平台组成。调试时先分别验证电机方向、编码器计数、传感器电平和 IMU 航向角，再进行 1~5 圈循迹停车和视觉识别的联调。由于现场光照、赛道黑线宽度和轮胎摩擦会影响控制效果，循迹参数需要在实际场地上反复修正。

五、作品成效总结分析

5.1 系统测试性能指标

测试内容	测试方法	结果记录
第一题循迹一圈	选择对应模式并启动，观察是否沿线绕行并停车	已完成
圈数设定循迹	分别设定 1、2、3、4、5 圈，观察到圈后是否停车	均可完成并停车，时间待实测
颜色识别	分别放置红、绿、蓝目标并观察屏幕标注	识别成功
几何图形识别	放置圆形、矩形、三角形目标并观察框选结果	圆形、三角形已有结果图
七段数码管识别	拍摄七段数字，观察 DIGIT 输出	已有识别结果图

表 5-1 系统测试与完成情况

5.2 成效得失对比分析

本作品的主要成效是完成了循迹小车的完整控制链路，并将视觉识别模块独立调通。循迹部分已经能够完成第一题要求，说明传感器接线、PWM 控制、定时器调度和基础状态机设计有效；视觉部分能够在 K230 上实时显示颜色、形状和数字结果，说明阈值分割和区域统计方法适合本次任务。

不足之处主要集中在赛题扩展功能和性能指标量化。当前 1~5 圈设定运行与到圈停车已经完成，但行驶时间 $t \leq 20\text{ s}$ 、终点停车偏差 $\leq 2\text{ cm}$ 等指标还需要用秒表和尺量方式补充实测数据。赛题中的障碍物避让、障碍物距离显示、蜂鸣器频次区分、二维云台对准目标以及终点 2 cm 精准泊车还需要继续补充传感器、舵机控制和停车判据。后续可进一步降低急弯速度波动，记录每圈状态计数、航向角和 PWM 输出，用数据方式优化行驶时间与停车位置。

5.3 创新特色与展望

作品的特色在于把传统 8 路循迹控制与 K230 视觉识别并行完成：底盘端强调稳定、低延迟的闭环控制，并已经完成 1~5 圈设定停车；视觉端强调识别结果可视化和快速调参。后续若继续完善，可将 K230 的识别结果通过串口发送给 MSPMOG3507，使小车根据颜色或图形结果触发蜂鸣器提示和云台转向；也可以增加上位机日志记录，把每圈的状态计数、航向角和 PWM 输出保存下来，用数据方式继续压缩行驶时间。

六、附件材料

附件材料包括参赛队员信息、竞赛相关图纸、完整程序清单、作品图片和演示视频等。本初稿已整理主要源码路径、关键程序片段和 K230 识别结果图片，队员专业特长、作品实物照片和演示视频文件名仍需后续补充。

附件项	当前材料
程序清单	电赛小车终极版（最终版）/user、code、ml_libs；K230 图像识别/*.py
作品图片	K230 图像识别结果图片
演示视频	绕行特定圈数后停止视频

表 6-1 附件材料整理情况

参考资料

[1] 南京林业大学校内电子设计竞赛设计报告格式要求，2026。

[2] 《设计报告书写基本格式》，南京林业大学校内电子设计竞赛资料。

[3] Texas Instruments, MSPM0G3507 Microcontroller Documentation。

[4] Kendryte K230 SDK 与图像处理接口相关文档。

[5] 本作品 MSPM0G3507 循迹小车源码与 K230 图像识别源码。